

LegalQuery Assistant
A Context-Aware Legal Document Analysis Using RAG

A report submitted in partial fulfillment of the requirements for the Degree of

Bachelor of Technology

in

DATA SCIENCE

by

M. Srinija	2211CS030103
Srinethri Shinde	2211CS030153
V. Mamatha	2211CS030168

Under the esteemed guidance of

Dr.K.Srikanth

(Associate Professor)



Department of Data Science

School of Engineering

MALLA REDDY UNIVERSITY

Maisammaguda, Dulapally, Hyderabad, Telangana 500100

2026

LegalQuery Assistant
A Context-Aware Legal Document Analysis Using Deep Learning

A report submitted in partial fulfillment of the requirements for the Degree of

Bachelor of Technology

in

DATA SCIENCE

by

M. Srinija	2211CS030103
Srinethri Shinde	2211CS030153
V. Mamatha	2211CS030168

Under the esteemed guidance of

Dr.K.Srikanth

(Associate Professor)



Department of Data Science

School of Engineering

MALLA REDDY UNIVERSITY

Maisammaguda, Dulapally, Hyderabad, Telangana 500100

2026



Department of Data Science

CERTIFICATE

This is to certify that the project report entitled “**LegalQuery Assistant**”, submitted by by **Ms. M. Srinija (2211CS030103), Ms. Srinethri Shinde (2211CS030153), Ms. V. Mamatha (2211CS030168)**, towards the partial fulfillment for the award of Bachelor’s Degree in B.Tech from the Department of Data science, Malla Reddy University, Hyderabad, is a record of bonafide work done by them. The results embodied in the work are not submitted to any other University or Institute for award of any degree or diploma.

Internal Guide:

Dr.K.Srikanth

Associate Professor

Dean of Data Science

Dr. G.S. Naveen Kumar

External Examiner

DECLARATION

We hereby declare that the project report entitled “**LegalQuery Assistant**” has been carried out by us and this work has been submitted to the Department of Data Science, Malla Reddy University, and Hyderabad in partial fulfillment of the requirements for the award of degree of Bachelor of Technology. We further declare that this project work has not been submitted in full or part for the award of any other degree in any other educational institutions.

Place: Hyderabad

Date:

M.Srinja	2211CS030103
Srinethri Shinde	2211CS030153
V.Mamatha	2211CS030168

ACKNOWLEDGEMENT

We extend our sincere gratitude to all those who have contributed to the completion of this project report. Firstly, We would like to extend our gratitude to **Dr. V. S. K Reddy, Vice-Chancellor**, for his visionary leadership and unwavering commitment to academic excellence. We would also like to express my deepest appreciation to our project guide **Dr.K.Srikanth Associate Professor**, whose invaluable guidance, insightful feedback, and unwavering support have been instrumental throughout the course of this project for successful outcomes.

We are also grateful to **Dr. G.S. Naveen Kumar, Dean of Data Science**, for providing us with the necessary resources and facilities to carry out this project.

We extend our sincere gratitude to **Mr.K Mahesh**, our Project Coordinator, for their valuable suggestions, continuous support, and motivation throughout the course of this project.

We are deeply indebted to all of them for their support, encouragement, and guidance, without which this project would not have been possible.

M.Srinija - 2211CS030103

Srinethri Shinde - 2211CS030153

V.Mamatha - 2211CS03016

ABSTRACT

Access to legal information is often complex and time-consuming for individuals who do not possess formal legal knowledge. Legal documents such as statutes, policies, and regulations are typically lengthy, written in highly technical language, and difficult for the general public to interpret. As a result, people frequently struggle to obtain clear answers to legal questions, leading to misinformation, misunderstanding of rights, or delayed decision-making. A Legal Query Assistant aims to address this challenge by providing an intelligent system that can understand legal queries and generate accurate, context-aware responses derived from relevant legal documents.

The system is designed to assist users in navigating legal information by enabling natural language interaction. Instead of manually searching through large volumes of legal text, users can submit questions in plain language and receive concise explanations based on the most relevant sections of legal documents. By analyzing and interpreting legal content, the assistant helps bridge the gap between complex legal language and everyday understanding.

The Legal Query Assistant enhances accessibility to legal knowledge by simplifying information retrieval and improving the efficiency of legal research. It can support students, researchers, legal professionals, and individuals seeking clarity on legal matters by providing structured and relevant information. Furthermore, such systems can contribute to improved legal awareness by making legal resources easier to explore and comprehend.

This Legal Query Assistant represents an innovative approach to legal information access, combining intelligent text understanding and knowledge retrieval to deliver meaningful responses. By transforming complex legal documents into accessible insights, the system contributes toward making legal information more transparent, approachable, and useful for a wider audience.

CONTENTS

Certificate		i
Declaration		ii
Acknowledgement		iii
Abstract		iv
Contents		v
List of Figures		vi
List of tables		vii
Chapter 1	Introduction	1
	1.1 Background Of Project	1
	1.2 Problem Statement	1
	1.3 Motivation	2
	1.4 Objectives of the Project	3
Chapter 2	Literature Review	4
Chapter 3	Proposed Methodology	7
	3.1 System Overview	7
	3.2 Tools and Technologies Used	8
	3.3 System Architecture	9
	3.3.1 Document Ingestion and Text Extraction	9
	3.3.2 Document Chunking and Structuring	10
	3.3.3 Embedding Generation	11
	3.3.4 Vector Storage and Retrieval	12
	3.3.5 Language Model Processing and Response Generation	14
	3.4 Frontend Interface and User Interaction	15
	3.5 API Endpoints	16
	3.6 Prompt Engineering Strategy	17
Chapter 4	Results and Discussions	19
Chapter 5	Conclusion and Future Scope of Study	23
Appendix	A. References	25
	B. Source code	26

LIST OF FIGURES

Figure	Title	Page
3.1	Document Processing Flow	11
3.2	Query Processing Flow	13
3.3	Response Generation Flow	15
4.1	Main Application Interface	19
4.2	Document Upload Interface	20
4.3	Document Successfully Uploaded	20
4.4	User Submitting a Query	21
4.5	System Processing the Query	21
4.6	Generated Response from the System	22
4.7	Final Response with Query Constraints	22

LIST OF TABLES

Figure	Title	Page
3.1	Document endpoints of backend	16
3.2	Collection endpoints of backend	16
3.3	Query endpoints of backend	17
3.4	System endpoints of backend	17

Chapter 1

Introduction

1.1 Background of Project

Legal systems across the world rely heavily on large volumes of documents such as statutes, regulations, policies, and judicial decisions. These documents contain important information that governs rights, responsibilities, and legal procedures. However, legal texts are typically written in complex language and structured in lengthy formats that are difficult for the general public to understand. As a result, individuals who do not possess formal legal training often find it challenging to interpret legal documents or locate the specific information relevant to their queries.

Traditionally, people depend on legal professionals, extensive manual research, or online searches to obtain answers to legal questions. While these methods can be effective, they are often time-consuming and may require specialized knowledge to navigate legal terminology and document structures. With the increasing availability of digital legal resources, the volume of legal information has grown significantly, making it even more difficult for users to efficiently identify relevant content.

Advancements in artificial intelligence and natural language processing have created opportunities to improve access to legal information. Intelligent systems can analyze large collections of documents and interpret user queries written in natural language. By retrieving relevant information and presenting it in a simplified manner, such systems can assist users in understanding legal concepts more effectively. This has led to the development of automated legal assistance tools that aim to make legal knowledge more accessible, organized, and easier to navigate for a broader range of users.

1.2 Problem Statement

Accessing accurate and relevant legal information remains a significant challenge for many individuals. Legal documents such as acts, regulations, and policies are often lengthy and written using complex legal terminology, making them difficult for non-experts to understand. People seeking answers to legal questions frequently have to search through multiple sources, read extensive documents, and interpret technical language in order to find the required information. This

process can be time-consuming and confusing, especially for individuals who lack formal legal knowledge.

Another major issue is the difficulty in identifying the most relevant sections of legal documents. Traditional search methods often rely on keyword matching, which may return large volumes of results without clearly indicating which sections actually address the user's query. As a result, users may struggle to locate precise and reliable information within large collections of legal texts.

The increasing availability of digital legal documents has further intensified this problem. Although more legal resources are accessible online, the absence of intelligent tools to interpret and organize this information limits their usefulness for the general public. There is therefore a growing need for a system that can understand legal queries expressed in natural language and retrieve relevant information from legal documents efficiently. Such a system can help simplify legal information access and improve users' ability to obtain meaningful and understandable answers.

1.3 Motivation

The motivation for developing a Legal Query Assistant arises from the growing need to make legal information more accessible and understandable to a wider audience. Legal knowledge plays an important role in everyday life, as laws and regulations influence various aspects of society such as employment, property, contracts, and personal rights. However, many individuals find it difficult to interpret legal documents due to their complex language, technical terminology, and lengthy structure. This gap between legal information and user understanding often prevents people from obtaining clear answers to their legal questions.

Another motivating factor is the increasing availability of digital legal resources. While numerous legal documents are now accessible online, simply providing access to these documents does not guarantee that users can easily extract the information they need. Users often spend considerable time searching through multiple sources, which can lead to confusion and inefficient information retrieval. An intelligent system capable of analyzing legal documents and responding to queries can significantly improve the efficiency of this process.

Advancements in artificial intelligence and natural language processing have created opportunities to build systems that can understand user queries and retrieve relevant information from large document collections. The project aims to explore how intelligent information retrieval can assist users in obtaining clear and relevant legal insights.

1.4 Objectives of the Project

The primary objective of the Legal Query Assistant project is to develop an intelligent system that can assist users in obtaining relevant legal information from large collections of legal documents in a simple and efficient manner. The system aims to reduce the complexity involved in manually searching and interpreting legal texts by enabling users to interact with legal information through natural language queries. By integrating intelligent information retrieval techniques, the project seeks to improve accessibility and usability of legal resources for individuals with varying levels of legal knowledge.

The key objectives of this project include:

- To design a system that allows users to submit legal queries using natural language.
- To enable efficient processing and analysis of legal documents for information retrieval.
- To identify and retrieve relevant sections of legal documents related to user queries.
- To provide concise and contextually relevant responses based on available legal information.
- To improve accessibility to legal knowledge for users without formal legal expertise.
- To explore the use of intelligent text analysis techniques for enhancing legal information retrieval.
- To develop a system that simplifies the process of understanding complex legal content.

Chapter 2

Literature Review

1. “Attention Is All You Need” by A. Vaswani, N. Shazeer, N. Parmar, et al. (2017)

This research introduces the Transformer architecture, a deep learning model that relies entirely on attention mechanisms for processing sequential data. Unlike traditional recurrent neural networks (RNNs) or convolutional neural networks (CNNs), the Transformer eliminates the need for sequential processing by using self-attention to capture relationships between words within a sentence simultaneously. This mechanism allows the model to understand long-range dependencies in text more effectively while also improving training efficiency.

The study demonstrates that attention-based models significantly improve performance in natural language processing tasks such as machine translation, text summarization, and language understanding. By enabling models to focus on the most relevant parts of input text, the Transformer architecture allows more accurate contextual interpretation of language. This work laid the foundation for modern language models such as BERT, GPT, and other large language models that power many AI-driven applications today.

The concepts introduced in this research are highly relevant for systems that require contextual understanding of text, including intelligent query assistants. Systems that process user queries and generate responses depend heavily on Transformer-based architectures to interpret language context, retrieve relevant information, and produce coherent outputs.

2. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks” by P. Lewis, E. Perez, A. Piktus, et al. (2020)

This study presents the Retrieval-Augmented Generation (RAG) framework, which combines traditional information retrieval techniques with generative language models to improve the accuracy and reliability of responses in knowledge-intensive tasks. The proposed approach enables a language model to retrieve relevant documents from a knowledge base and use them as contextual information while generating answers.

One of the key limitations of purely generative language models is that they rely only on their training data and internal parameters to produce responses. This can sometimes lead to incorrect or unsupported information. The RAG framework addresses this issue by grounding the generated responses in external knowledge sources, ensuring that the answers are supported by retrieved documents.

The research demonstrates that integrating retrieval mechanisms with generative models significantly improves performance in tasks such as open-domain question answering, knowledge retrieval, and fact-based text generation. By retrieving relevant context before generating responses, the model becomes more reliable and capable of handling queries related to large knowledge bases.

The RAG concept is particularly significant for systems that require accurate responses derived from large document collections, such as legal information retrieval systems. In legal applications, where accuracy and contextual grounding are critical, retrieval-augmented generation provides a practical approach for combining document search with intelligent response generation.

3. “Neural Legal Information Retrieval and Question Answering” by I. Chalkidis, I. Androutsopoulos, and N. Aletras (2019)

This research focuses on the application of neural network models for legal information retrieval and automated question answering. The study addresses the unique challenges associated with processing legal texts, including complex language structures, specialized terminology, and the hierarchical organization of legal documents.

Legal documents often contain long paragraphs, multiple clauses, and references to other legal provisions, which makes traditional keyword-based search methods insufficient for retrieving relevant information.

The authors propose the use of neural ranking models and machine learning techniques to improve the retrieval of relevant legal information from large collections of legal documents. These models analyze semantic relationships between queries and legal texts rather than relying solely on keyword matching. By learning contextual representations of legal language, neural models can better identify the sections of documents that contain answers to specific legal questions.

The findings suggest that neural models significantly enhance the effectiveness of legal question-answering systems by improving document ranking accuracy and contextual understanding. Such approaches enable automated systems to retrieve and interpret relevant legal information more efficiently.

The research contributes valuable insights into the development of intelligent systems designed to assist users in accessing and interpreting legal information. It highlights the importance of combining natural language processing, machine learning, and information retrieval techniques for building effective legal assistance tools.

Chapter 3

Proposed Methodology

3.1 System Overview

The Legal Query Assistant is an intelligent system designed to help users obtain relevant legal information from large collections of legal documents. The proposed system simplifies this process by allowing users to submit legal questions in natural language and receive clear responses derived from relevant legal documents.

The system follows a structured workflow that processes legal documents, converts them into searchable representations, and retrieves the most relevant information when a user submits a query. Legal documents such as PDFs or text files are first processed and divided into smaller segments to improve retrieval accuracy. These segments are then transformed into vector representations using embedding models, allowing the system to capture the semantic meaning of the text.

The generated embeddings are stored in a vector database, which enables efficient similarity-based search. When a user submits a legal query, the system converts the query into a vector representation and performs a semantic similarity search to identify the most relevant document sections. These retrieved sections serve as contextual information for a language model that generates the final response. The system therefore combines information retrieval and language generation techniques to produce accurate and context-aware answers.

The main components of the system include:

- **Legal Document Input:** Legal documents such as PDFs or text files are provided as the knowledge source.
- **Text Splitting:** Large documents are divided into smaller chunks to enable efficient processing and retrieval.
- **Embedding Generation:** Text chunks are converted into numerical vector representations that capture semantic meaning.

- **Vector Database Storage:** The generated embeddings are stored in a vector database for efficient indexing and retrieval.
- **Semantic Similarity Search:** User queries are compared with stored embeddings to identify relevant legal document sections.
- **Query and Context Integration:** The retrieved document content is combined with the user's query to form contextual input.
- **Language Model Processing:** An instruction-tuned language model generates responses based on the provided context.
- **Answer Generation:** The system produces a concise and contextually relevant answer to the user's legal query

3.2 Tools and Technologies Used

The Legal Query Assistant system is developed using several modern tools and technologies that support document processing, semantic search, and intelligent response generation. These technologies collectively enable efficient implementation of the retrieval-augmented generation architecture used in the system.

Python is used as the primary programming language for implementing the backend logic and machine learning components of the system. Python provides extensive libraries for natural language processing, machine learning, and API development, making it suitable for building AI-driven applications.

FastAPI is used as the backend web framework to build RESTful APIs for the system. It allows efficient communication between the frontend interface and backend pipelines, handling operations such as document upload, query processing, and retrieval of responses.

Typesense is used as the vector database for storing and retrieving document embeddings. It enables efficient semantic similarity search by indexing high-dimensional vector representations of document chunks.

SentenceTransformers is used for generating semantic embeddings from document text. These embeddings allow the system to capture contextual meaning and perform similarity-based retrieval of relevant document sections.

Tesseract OCR is used to extract text from scanned documents or image-based files. This ensures that documents without native text can still be processed by the system.

Groq API is used to access high-performance large language models for generating responses based on retrieved document context. The Groq provider enables fast inference and efficient language model processing within the system.

3.3 System Architecture

The Legal Query Assistant follows a modular system architecture designed to efficiently process legal documents and generate accurate responses to user queries. The architecture is based on a sequence of interconnected pipelines that transform raw legal documents into structured knowledge and use this information to generate context-aware responses. Each pipeline performs a specific task within the overall workflow, ensuring that the system remains scalable, maintainable, and easy to extend.

The architecture consists of five main processing pipelines: Document Ingestion and Text Extraction, Document Chunking and Structuring, Embedding Generation, Vector Storage and Retrieval, and Language Model Processing. These pipelines collectively enable the system to process legal documents, store them in a searchable format, and generate answers based on relevant information retrieved from the document collection.

3.3.1 Document Ingestion and Text Extraction

The first stage of the system architecture is the document ingestion and text extraction pipeline. This component is responsible for accepting legal documents as input and converting them into machine-readable text. Legal documents may exist in various formats such as PDF files, Word documents, images, or plain text files. Since these formats store information differently, the system must first detect the document type and apply the appropriate method for extracting text.

For documents containing native text, such as digital PDFs or text files, the system directly extracts the textual content. However, many legal documents are available as scanned images or scanned

PDFs that contain text embedded within images. In such cases, the system applies Optical Character Recognition (OCR) techniques to identify and extract the text from the images. This ensures that both digital and scanned legal documents can be processed by the system.

During the extraction process, the system also gathers additional information such as the number of pages in the document, the proportion of text extracted through OCR, and any errors encountered during processing. The extracted text is then prepared for the next stage of processing, where it will be structured and divided into smaller segments for efficient retrieval.

3.3.2 Document Chunking and Structuring

Legal documents are often very long and contain multiple sections, subsections, and clauses. Processing entire documents as a single unit would reduce retrieval accuracy and make it difficult to identify the exact portions of text that answer a user's query. To address this issue, the system uses a document chunking and structuring pipeline that divides large legal texts into smaller and manageable segments.

The chunking process begins by analyzing the structure of the document to detect elements such as headings, numbered sections, paragraphs, and clauses. These structural indicators help preserve the hierarchical organization of the document while splitting it into chunks. Maintaining this structure is important because legal meaning often depends on the relationship between sections and subsections.

Once the document structure is detected, the system applies appropriate chunking strategies to divide the text into meaningful segments. Each chunk typically contains a limited number of sentences or paragraphs while maintaining contextual coherence. In addition to the text content, each chunk is associated with metadata such as section hierarchy, source document name, and position within the document.

This structured representation allows the system to retrieve relevant portions of legal documents more precisely during the query stage. By breaking large documents into smaller units while preserving their structural context, the chunking pipeline improves the efficiency and accuracy of the overall information retrieval process.

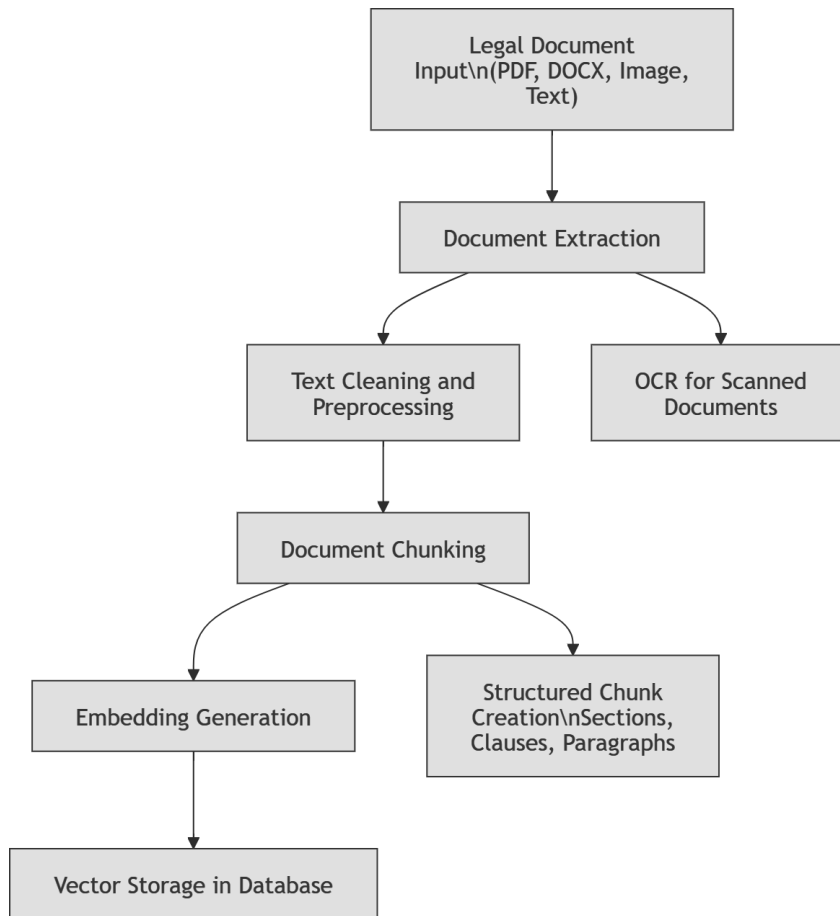


Fig 3.1: Document Processing Flow

3.3.3 Embedding Generation

After the documents have been divided into chunks, the next step is to convert these text segments into numerical vector representations known as embeddings. Embeddings capture the semantic meaning of text by representing it as a set of numerical values in a multi-dimensional vector space. This representation enables the system to compare the similarity between different pieces of text based on their meaning rather than relying solely on keyword matching.

During this stage, each chunk generated from the previous pipeline is processed by an embedding model. The model analyzes the textual content and generates a corresponding vector representation. These vectors allow the system to understand relationships between words, phrases, and sentences in a way that reflects their contextual meaning.

The embedding generation process uses transformer-based sentence embedding models to convert text chunks into semantic vector representations. The primary embedding model used in the system

is **BAAI/bge-base-en-v1.5**, which produces embeddings with a dimensionality of 768. This model belongs to the BGE (BAAI General Embedding) family developed for high-quality semantic retrieval tasks. It is trained on large-scale datasets to capture contextual meaning and relationships between words and sentences, making it highly effective for applications such as semantic search, question answering, and retrieval-augmented generation systems. The model is particularly well suited for comparing textual similarity between user queries and document content, enabling the system to retrieve the most relevant legal text segments even when the wording of the query differs from the original document.

To ensure system reliability and robustness, a fallback mechanism is implemented using the **all-MiniLM-L6-v2** embedding model. This model is part of the SentenceTransformers family and is widely used for generating compact yet effective sentence embeddings. Although it is smaller and computationally lighter than many large embedding models, it still provides strong performance for semantic similarity tasks. The model generates embeddings of the same dimensional size (768), allowing seamless integration within the system if the primary model is unavailable. If the primary embedding model fails to load or initialize, the system automatically switches to the fallback model, ensuring that the embedding generation process continues without interruption. This fallback strategy improves system stability and guarantees consistent performance during document processing and query retrieval operations.

Embedding generation plays a crucial role in enabling semantic search within the system. When embeddings are created for document chunks, the system can later generate a similar embedding for a user's query and measure how closely it matches the stored document embeddings. This approach allows the system to retrieve relevant legal information even when the query uses different wording from the original document text.

In addition to the embedding vectors, the system also preserves associated metadata such as the source document, chunk hierarchy, and additional contextual information. This metadata is later used during the retrieval stage to provide more informative responses to user queries.

3.3.4 Vector Storage and Retrieval

Once embeddings have been generated for all document chunks, they must be stored in a system that allows efficient retrieval based on similarity. This is handled by the vector storage and retrieval pipeline. The embeddings are stored in a vector database that is specifically designed to support

high-dimensional vector search.

In the vector database, each stored record contains the embedding vector along with its corresponding metadata and original text content. The database indexes these vectors in a way that allows rapid similarity comparisons between vectors.

When a user submits a query, the system converts the query into an embedding using the same embedding model used during document processing. The vector database then performs a similarity search between the query vector and the stored document embeddings. The database returns the chunks that are most semantically similar to the user's query.

The retrieved chunks represent the most relevant sections of the legal documents that are likely to contain the information needed to answer the query. These chunks are then passed to the next stage of the system, where they are used as contextual input for generating the final response.

The system uses **Typesense** as the vector storage and retrieval engine. The Vector Storage Pipeline is implemented through a TypesenseVectorStore wrapper that manages vector indexing and similarity search operations. All generated embeddings are stored along with their associated metadata, such as source document name, section hierarchy, and chunk identifiers. Typesense enables efficient semantic similarity search by comparing query embeddings with stored document embeddings and returning the most relevant document segments.

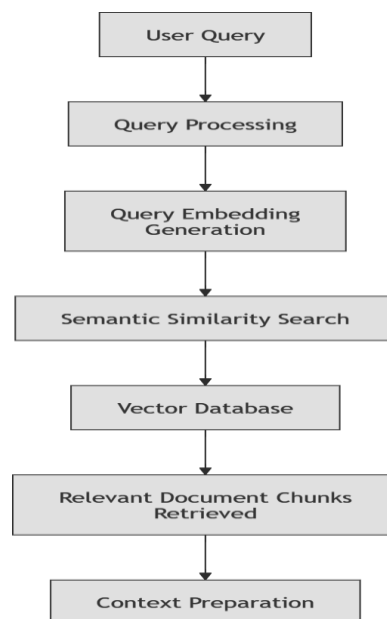


Fig 3.2: Query Processing Flow

3.3.5 Language Model Processing and Response Generation

The final stage of the system architecture involves generating a response based on the information retrieved from the vector database. This process uses a language model capable of understanding natural language queries and producing coherent textual responses.

In this stage, the system combines the user's query with the retrieved document chunks to create a contextual input. This input is then passed to a language model that analyzes the provided information and generates a response that addresses the user's query. By grounding the response in retrieved legal document content, the system ensures that the generated answers are relevant and context-aware.

The language model also considers the conversational context when users ask follow-up questions. This enables the system to maintain continuity in interactions and provide more refined responses during ongoing conversations.

The response generation component uses a large language model provided through the **Groq API**, which enables fast and efficient inference for natural language generation tasks. The Groq-based language model processes the user query along with retrieved document context to generate relevant and coherent responses.

To support multi-turn interactions, the system maintains conversation memory for each user session. Conversation history is stored as a list of ConversationMessage objects within each LLM pipeline instance. This allows the system to retain previous interactions and provide context-aware responses for follow-up queries while maintaining isolation between different user conversations.

The final output of the system is a concise and informative answer derived from relevant legal documents. This approach allows the Legal Query Assistant to bridge the gap between complex legal texts and user-friendly explanations, thereby improving accessibility to legal knowledge.

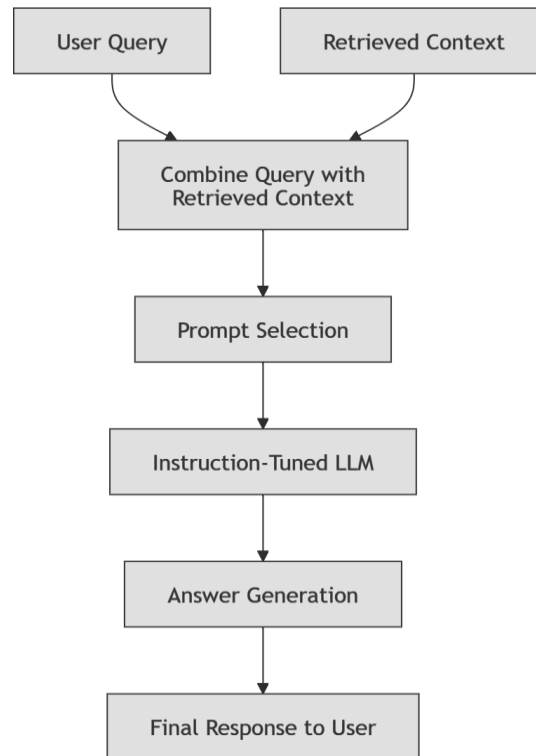


Fig 3.3: Response Generation Flow

3.4 Frontend Interface and User Interaction

The frontend interface provides the interaction layer between the user and the Legal Query Assistant system. It enables users to interact with the system in a simple and intuitive manner without requiring technical knowledge of the underlying processes. Through the interface, users can submit legal queries, upload legal documents, and receive responses generated by the backend processing system. The interface is designed to support natural language interaction, allowing users to ask questions in a conversational format.

Users can enter legal queries through the query input field available in the interface. Once a query is submitted, the frontend sends the request to the backend server through API calls. The backend processes the query using retrieval and language generation pipelines and returns a response derived from relevant legal document content. The frontend then displays the generated response in a conversational format, making it easier for users to read and understand the information.

In addition to submitting queries, the interface also allows users to upload legal documents such as PDFs, text files, or other supported formats. These uploaded documents are sent to the backend where they undergo document processing, including text extraction, chunking, embedding

generation, and storage in the vector database. This feature enables the system to expand its knowledge base and retrieve information from newly uploaded legal documents.

The interface also supports follow-up queries, allowing users to continue conversations and obtain additional clarifications. This interaction mechanism ensures seamless communication between the user and the system while enabling efficient processing and retrieval of legal information.

3.5 API Endpoints

The backend exposes a set of REST API endpoints that allow the frontend interface to interact with the document processing and query pipelines. These endpoints are organized into three main routers responsible for document management, collection management, and query processing.

Document Endpoints

Method	Endpoint	Purpose
POST	/documents/upload	Upload a document and return a document ID
POST	/documents/{id}/process	Execute the full document processing pipeline
GET	/documents/{id}/status	Check the ingestion status of the document

Table 3.1 Document endpoints of backend

Collection Endpoints

Method	Endpoint	Purpose
POST	/collections	Create a new vector collection
GET	/collections	List all available collections
GET	/collections/{name}	Inspect a collection
DELETE	/collections/{name}	Delete a collection

Table 3.2 Collection endpoints of backend

Query Endpoints

Method	Endpoint	Purpose
POST	/query	Perform a RAG query with optional conversation ID
GET	/query/conversations/{id}	Retrieve conversation history
DELETE	/query/conversations/{id}	Clear conversation history

Table 3.3 Query endpoints of backend

System Endpoints

Method	Endpoint	Purpose
GET	/health	Check pipeline health status
GET	/status	Retrieve overall system status

Table 3.4 System endpoints of backend

3.6 Prompt Engineering Strategy

Prompt engineering plays a crucial role in guiding the language model to generate accurate and context-aware responses. In the Legal Query Assistant system, prompt templates are carefully designed to structure the input provided to the language model. Instead of sending the user query directly to the model, the system combines the user query with relevant document context and a predefined prompt template. This structured prompt helps the model understand the task and generate responses that are grounded in retrieved legal information.

The system uses multiple prompt templates that are selected dynamically based on the type of user query and the quality of the retrieved context. Queries are first classified into categories such as **new queries**, **follow-up queries**, or **clarification requests**. In addition, the retrieved document context is evaluated using similarity scores to determine its relevance to the query.

Based on these factors, the system selects one of several prompt templates to guide the response generation process. For example, when highly relevant context is retrieved, the prompt instructs the model to generate an answer directly using the provided legal information. If the retrieved context has lower relevance, the prompt encourages the model to provide a cautious response and indicate possible uncertainty.

This adaptive prompt selection strategy helps improve response reliability and reduces the risk of generating unsupported information. By tailoring prompts according to query type and context quality, the system ensures that the language model produces responses that are both informative and aligned with the available legal document content.

Chapter 4

Results and Discussions

The below is the main interface of the Legal Query Assistant, LEXORA application. The interface contains a document panel on the left side where uploaded documents are displayed. The central section serves as the interaction area where users can ask questions related to the uploaded documents. The query input field is located at the bottom of the screen, allowing users to type legal queries in natural language. This interface acts as the primary interaction layer between the user and the backend system.



Fig 4.1: Main Application Interface

This figure shows the document upload interface of the Legal Query Assistant system. The interface allows users to select a legal document from their local system using the “**Choose File**” option. Once a file is selected, the user can upload it to the system using the **Upload** button. This functionality enables users to add legal documents to the system's knowledge base. After uploading, the document is sent to the backend where it undergoes processing such as text extraction, chunking, embedding generation, and storage in the vector database.



Fig 4.2: Document Upload Interface

This figure shows the system after a document has been successfully uploaded. The uploaded file appears in the **Documents** panel on the left side, indicating that the document is now available for query processing. At this stage, the document has already been processed by the backend pipelines and stored in the vector database, allowing the system to retrieve relevant information from the document when a user submits a query.

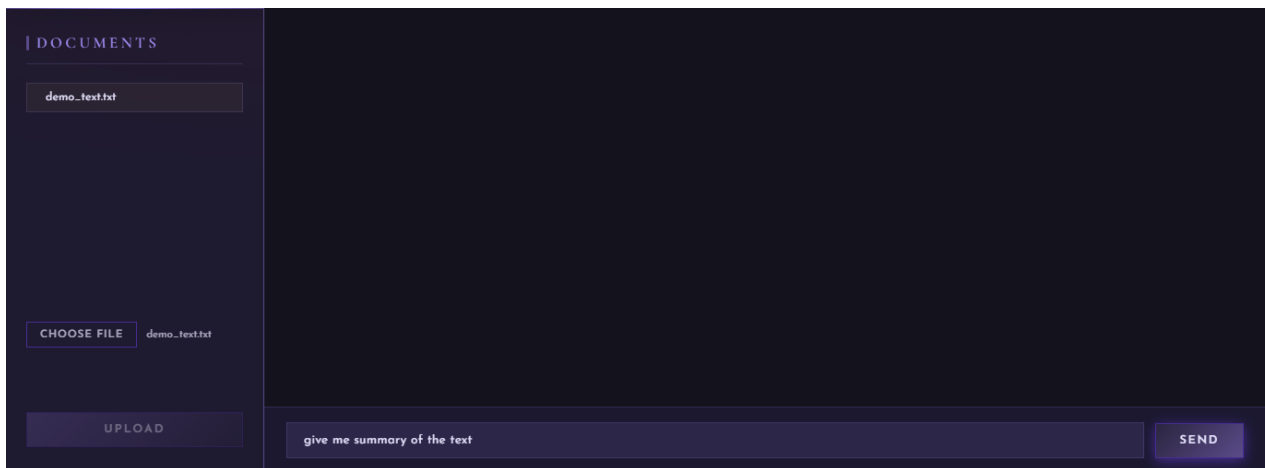


Fig 4.3: Document Successfully Uploaded

This figure demonstrates the moment when the user submits a query to the system. The user asks a question related to the uploaded document through the query input field. Once the query is submitted, it is sent to the backend system where it undergoes query processing, embedding generation, and semantic similarity search to retrieve relevant document sections.

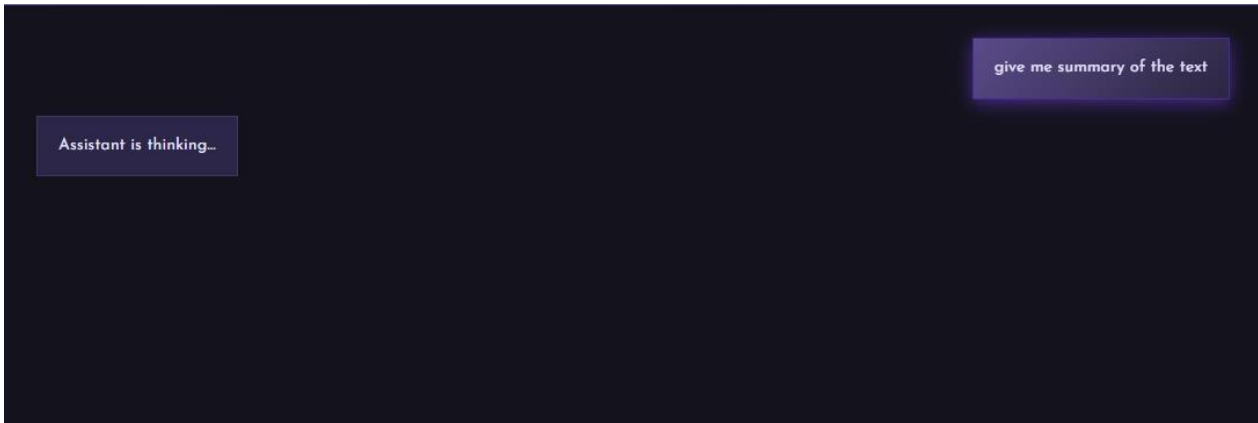


Fig 4.4: User Submitting a Query

This figure shows the system while processing the user query. The interface displays the message “**Assistant is thinking...**”, indicating that the backend is performing several operations such as generating the query embedding, retrieving relevant document chunks from the vector database, and preparing the context required for response generation using the language model.

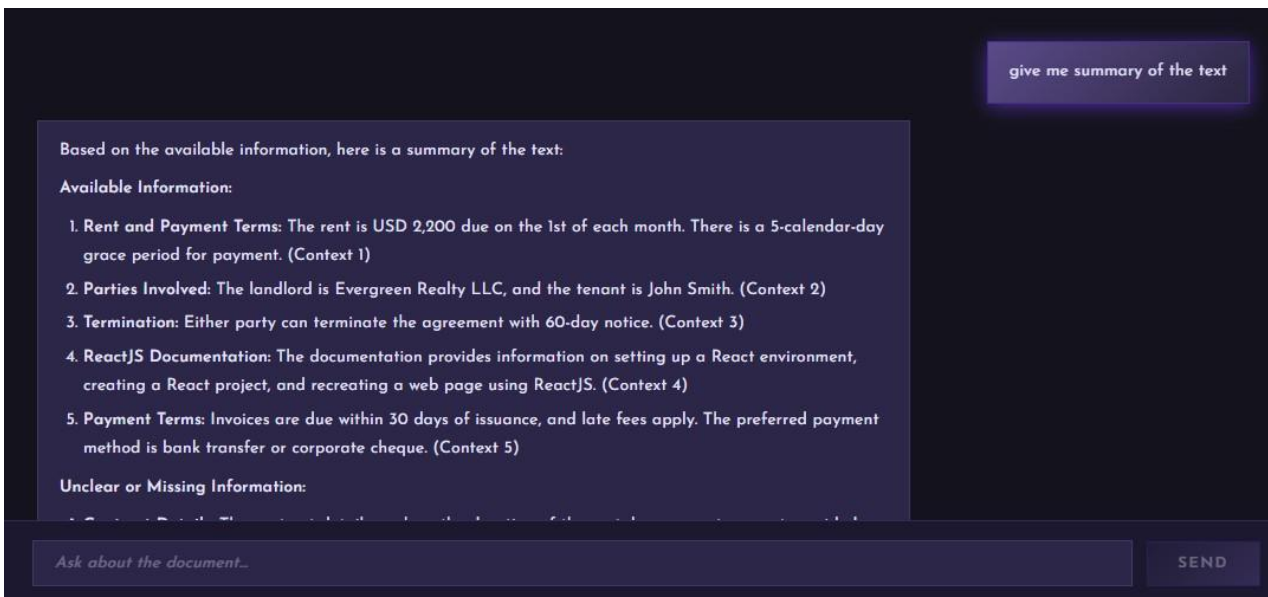


Fig 4.5: System Processing the Query

This figure shows the response generated by the Legal Query Assistant after processing the query. The system retrieves relevant sections from the document and generates a summary based on the available information. The response includes contextual references from the document, demonstrating how the system grounds its answers in retrieved document content.

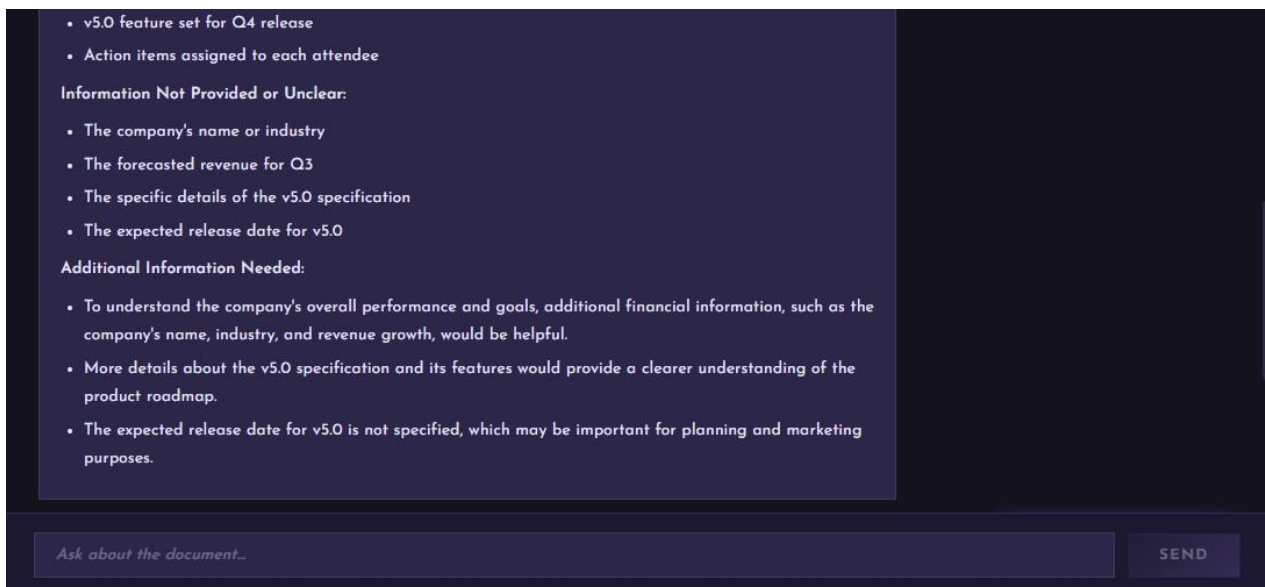


Fig 4.6: Generated Response from the System

This figure illustrates a detailed response generated by the system using contextual information retrieved from the uploaded document. The system provides structured output including available information, unclear points, and additional information required. This demonstrates the system's ability to analyze document content and generate meaningful insights based on retrieved context. This figure shows the final response generated by the system when the user provides additional instructions such as limiting the answer length. The system adapts the generated response according to the query requirements, producing a concise answer within the specified word limit. This demonstrates the flexibility of the system in handling user-specific query instructions.

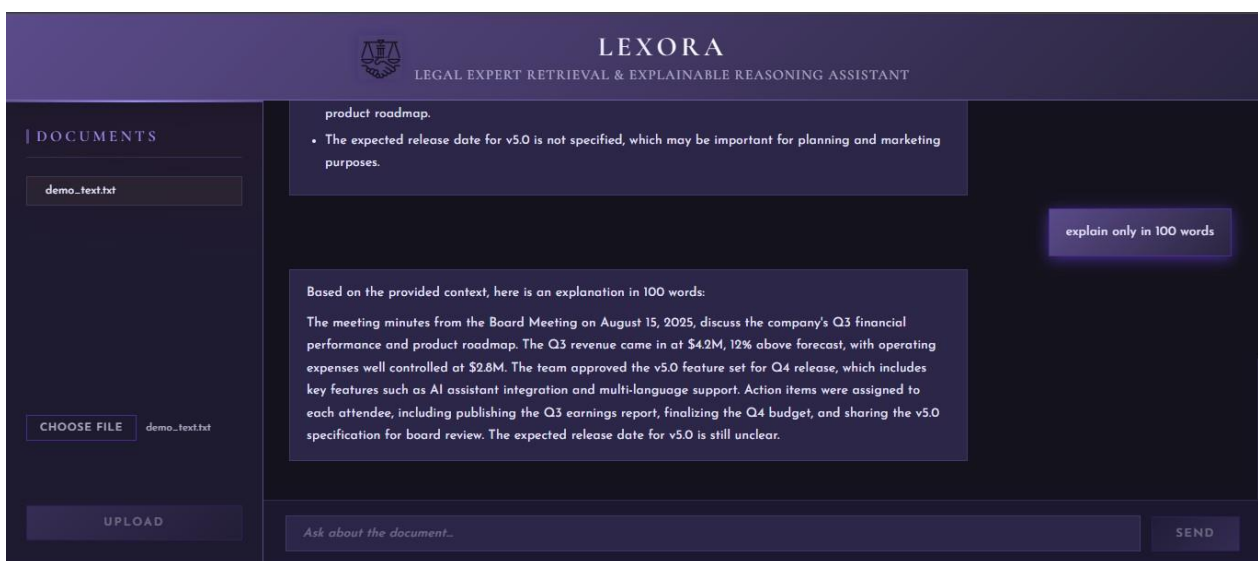


Fig 4.7: Final Response with Query Constraints

Chapter 5

Conclusion

The Legal Query Assistant was developed to improve accessibility to legal information by enabling users to interact with legal documents through natural language queries. Legal documents are often lengthy and written in complex terminology, making it difficult for individuals without legal expertise to locate relevant information. The proposed system addresses this challenge by providing an intelligent platform that processes legal documents, retrieves relevant information, and generates context-aware responses to user queries.

The system integrates document processing, semantic search, and language generation techniques to support efficient legal information retrieval. Legal documents uploaded to the system are processed through multiple stages, including text extraction, document chunking, embedding generation, and storage in a vector database. This structured processing enables the system to organize large volumes of legal text into searchable units, improving the accuracy of information retrieval.

When a user submits a query, the system analyzes the question and performs a semantic similarity search to identify relevant sections of the processed legal documents. These retrieved sections serve as contextual input for a language model, which generates responses based on the information available in the documents. This approach ensures that the answers produced by the system are grounded in relevant legal content rather than relying solely on generative language capabilities.

The implementation of the Legal Query Assistant demonstrates how intelligent information retrieval and natural language processing techniques can enhance the usability of legal resources. The system allows users to upload legal documents, ask questions related to the content, and receive meaningful responses in a conversational format. This significantly reduces the effort required to manually search through large legal documents.

Overall, the proposed system provides an effective solution for simplifying access to legal information. By combining document analysis, semantic retrieval, and automated response generation, the Legal Query Assistant contributes toward making legal knowledge more accessible, efficient, and user-friendly for a wide range of users.

Future Scope

Although the Legal Query Assistant provides an effective approach for retrieving and explaining legal information, several improvements can be made to enhance the system's capabilities and scalability in future developments. One important area for improvement is the integration of a persistent database to store conversation history and document metadata. Currently, the system stores this information in memory, which means that the data may be lost if the system restarts. Implementing persistent storage solutions such as relational or NoSQL databases would improve system reliability and long-term usability.

Another potential enhancement involves expanding the range of legal documents supported by the system. Future versions could incorporate larger collections of legal resources such as case laws, judicial decisions, government regulations, and legal commentaries. Integrating multiple legal knowledge sources would enable the system to provide more comprehensive and accurate responses to user queries.

The system could also be improved by incorporating advanced natural language understanding capabilities to better interpret complex legal questions and provide more precise answers. Additionally, multilingual support could be implemented to allow users to submit queries and receive responses in different languages, increasing accessibility for a wider audience.

Further improvements may include integrating citation mechanisms that clearly reference the legal sources used in generating responses. Such enhancements would improve transparency and reliability, making the Legal Query Assistant a more robust tool for legal research and information retrieval in real-world applications.

Appendix

A. References

- Vaswani, A., Shazeer, N., Parmar, N., et al., “Attention Is All You Need,” Advances in Neural Information Processing Systems (NeurIPS), 2017.
- Lewis, P., Perez, E., Piktus, A., et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” Advances in Neural Information Processing Systems (NeurIPS), 2020.
- Chalkidis, I., Androutsopoulos, I., and Aletras, N., “Neural Legal Information Retrieval and Question Answering,” Proceedings of the Natural Legal Language Processing Workshop (NLLP), 2019.

B. Source Code:

Backend:

Main.py

```
"""
```

```
RAG API - Main Application
```

```
FastAPI application connecting all 5 pipelines.
```

```
Endpoints:
```

- POST /documents/upload - Upload document
- POST /documents/{id}/process - Process document
- GET /documents/{id}/status - Get processing status
- POST /collections/ - Create collection
- GET /collections/ - List collections
- GET /collections/{name} - Get collection info
- DELETE /collections/{name} - Delete collection
- POST /query/ - Query RAG system
- GET /query/conversations/{id} - Get conversation history
- DELETE /query/conversations/{id} - Clear conversation
- GET /health - Health check
- GET /status - System status

```
"""
```

```
import logging
```

```
from fastapi import FastAPI, HTTPException
```

```
from fastapi.middleware.cors import CORSMiddleware
```

```
from contextlib import asynccontextmanager
```

```
from config.settings import RAGConfig, get_pipeline_manager
```

```
from routers import documents, query, collections
```

```
from schemas.models import HealthResponse, SystemStatusResponse
```

```
# Configure logging
```

```
logging.basicConfig(
```

```
    level=logging.INFO,
```

```
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
```

```
)
```

```
log = logging.getLogger(__name__)
```

```
import dotenv
```

```
dotenv.load_dotenv()
```

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    """
    Startup and shutdown events.
    """
    # Startup
    log.info("Starting RAG API...")

    # Validate configuration
    config_errors = RAGConfig.validate()
    if config_errors:
        log.error(f"Configuration errors: {config_errors}")
        raise ValueError(f"Invalid configuration: {config_errors}")

    # Initialize pipelines
    pm = get_pipeline_manager()

    try:
        pm.initialize_all()
        log.info("✓ All pipelines initialized successfully")
    except Exception as e:
        log.error(f"Failed to initialize pipelines: {e}")
        raise

    log.info(f"RAG API started on {RAGConfig.API_HOST}:{RAGConfig.API_PORT}")

    yield

    # Shutdown
    log.info("Shutting down RAG API...")

# Create FastAPI app
app = FastAPI(
    title="Legal RAG API",
    description="Production-ready RAG system for legal documents",
    version=RAGConfig.API_VERSION,
    lifespan=lifespan,
)

# CORS middleware
```

```

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], # Configure appropriately for production
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Include routers
app.include_router(documents.router)
app.include_router(collections.router)
app.include_router(query.router)

#
=====

# Health & Status Endpoints
#
=====

@app.get("/", tags=["Root"])
async def root():
    """API root endpoint."""
    return {
        "message": "Legal RAG API",
        "version": RAGConfig.API_VERSION,
        "docs": "/docs",
        "health": "/health",
    }

@app.get("/health", response_model=HealthResponse, tags=["Health"])
async def health_check():
    """
    Health check endpoint.
    Returns status of all pipelines.
    """
    pm = get_pipeline_manager()
    pipeline_status = pm.health_check()

    all_healthy = all(pipeline_status.values())

```

```

return HealthResponse(
    status="healthy" if all_healthy else "degraded",
    version=RAGConfig.API_VERSION,
    pipelines=pipeline_status,
    message="All systems operational" if all_healthy else "Some pipelines unavailable",
)

```

```

@app.get("/status", response_model=SystemStatusResponse, tags=["Health"])

```

```

async def system_status():

```

```

    """

```

```

    Detailed system status.

```

```

    """

```

```

    pm = get_pipeline_manager()

```

```

    pipeline_status = pm.health_check()

```

```

    # Get collection stats

```

```

    try:

```

```

        collections = pm.storage.list_collections()

```

```

        total_collections = len(collections)

```

```

        total_documents = sum(c.num_documents for c in collections)

```

```

    except:

```

```

        total_collections = 0

```

```

        total_documents = 0

```

```

    return SystemStatusResponse(

```

```

        api_version=RAGConfig.API_VERSION,

```

```

        ingestion_ready=pipeline_status["ingestion"],

```

```

        chunker_ready=pipeline_status["chunker"],

```

```

        embedder_ready=pipeline_status["embedder"],

```

```

        storage_ready=pipeline_status["storage"],

```

```

        llm_ready=pipeline_status["llm"],

```

```

        total_collections=total_collections,

```

```

        total_documents=total_documents,

```

```

        embedding_model=RAGConfig.EMBEDDING_MODEL,

```

```

        embedding_dim=RAGConfig.EMBEDDING_DIM,

```

```

        llm_model=RAGConfig.LLM_MODEL,

```

```

    )

```

```

#

```

```

# Exception Handlers

```

#

```
from fastapi.responses import JSONResponse
```

```
@app.exception_handler(HTTPException)
async def http_exception_handler(request, exc):
    return JSONResponse(
        status_code=exc.status_code,
        content={"error": exc.detail},
    )
```

```
@app.exception_handler(Exception)
async def general_exception_handler(request, exc):
    return JSONResponse(
        status_code=500,
        content={"error": "Internal server error", "detail": str(exc)},
    )
```

```
@app.exception_handler(Exception)
async def general_exception_handler(request, exc):
    """Handle general exceptions."""
    log.error(f"Unhandled exception: {exc}")
    return {
        "error": "Internal server error",
        "detail": str(exc),
        "status_code": 500,
    }
```

```
if __name__ == "__main__":
    import uvicorn
```

```
    uvicorn.run(
        "rag_api.main:app",
        host=RAGConfig.API_HOST,
        port=RAGConfig.API_PORT,
        reload=True,
        log_level="info",
    )
```

Config/settings.py

```
"""
```

Configuration & Pipeline Initialization

Loads config from environment and initializes all 5 pipelines.

```
"""
```

```
import os
```

```
import logging
```

```
from typing import Optional
```

```
# Pipeline imports
```

```
import sys
```

```
sys.path.insert(0, "/home/claude")
```

```
from doc_extractor.pipeline import DocumentIngestionPipeline
```

```
from legal_rag_chunker.pipeline import HierarchicalChunkingPipeline
```

```
from embedding_pipeline.pipeline import EmbeddingPipeline, EmbeddingConfig
```

```
from vector_storage_pipeline.pipeline import VectorStoragePipeline, StorageConfig
```

```
from llm_pipeline.pipeline import LLMPipeline, LLMConfig
```

```
log = logging.getLogger(__name__)
```

```
import dotenv
```

```
dotenv.load_dotenv()
```

```
class RAGConfig:
```

```
    """
```

```
    Configuration for the complete RAG system.
```

```
    Loads from environment variables with sensible defaults.
```

```
    """
```

```
    # API
```

```
    API_VERSION: str = "1.0.0"
```

```
    API_HOST: str = os.getenv("API_HOST", "0.0.0.0")
```

```
    API_PORT: int = int(os.getenv("API_PORT", "8000"))
```

```
    # Embedding
```

```
    EMBEDDING_MODEL: str = os.getenv("EMBEDDING_MODEL", "BAAI/bge-base-en-v1.5")
```

```
    EMBEDDING_FALLBACK: str = os.getenv("EMBEDDING_FALLBACK", "sentence-transformers/all-MiniLM-L6-v2")
```

```

EMBEDDING_DIM: int = int(os.getenv("EMBEDDING_DIM", "768"))

# Vector Storage (Typesense)
TYPESENSE_HOST: str = os.getenv("TYPESENSE_HOST")
TYPESENSE_PORT: int = int(os.getenv("TYPESENSE_PORT"))
TYPESENSE_PROTOCOL: str = os.getenv("TYPESENSE_PROTOCOL")
TYPESENSE_API_KEY: str = os.getenv("TYPESENSE_API_KEY")

# LLM (Groq)
GROQ_API_KEY: str = os.getenv("GROQ_API_KEY")
LLM_MODEL: str = os.getenv("LLM_MODEL")
LLM_TEMPERATURE: float = float(os.getenv("LLM_TEMPERATURE", "0.1"))
LLM_MAX_TOKENS: int = int(os.getenv("LLM_MAX_TOKENS", "1024"))

# Processing
OCR_DPI: int = int(os.getenv("OCR_DPI", "300"))
BATCH_SIZE: int = int(os.getenv("BATCH_SIZE", "100"))
CONTEXT_THRESHOLD: float = float(os.getenv("CONTEXT_THRESHOLD", "0.3"))
MAX_CONTEXT_LENGTH: int = int(os.getenv("MAX_CONTEXT_LENGTH", "5"))

# File storage
UPLOAD_DIR: str = os.getenv("UPLOAD_DIR", "./uploads")

@classmethod
def validate(cls) -> list[str]:
    """Validate configuration."""
    errors = []

    if not cls.GROQ_API_KEY:
        errors.append("GROQ_API_KEY is required")

    if not cls.TYPESENSE_API_KEY:
        errors.append("TYPESENSE_API_KEY is required")

    return errors

class PipelineManager:
    """
    Manages all 5 pipelines as singletons.
    Initializes once at startup.
    """

    _instance: Optional['PipelineManager'] = None

```

```

def __new__(cls):
    if cls._instance is None:
        cls._instance = super().__new__(cls)
        cls._instance._initialized = False
    return cls._instance

def __init__(self):
    if self._initialized:
        return

    self._initialized = True

    # Pipeline instances
    self.ingestion: Optional[DocumentIngestionPipeline] = None
    self.chunker: Optional[HierarchicalChunkingPipeline] = None
    self.embedder: Optional[EmbeddingPipeline] = None
    self.storage: Optional[VectorStoragePipeline] = None
    self.llm: Optional[LLMPipeline] = None

    # Document and conversation storage (in-memory for now)
    self.documents: dict = {} # doc_id -> status info
    self.conversations: dict = {} # conversation_id -> history

    log.info("PipelineManager created")

def initialize_all(self):
    """Initialize all pipelines."""
    log.info("Initializing RAG pipelines...")

    try:
        # Stage 1: Document Ingestion
        self.ingestion = DocumentIngestionPipeline(
            ocr_dpi=RAGConfig.OCR_DPI
        )
        log.info("✓ Document Ingestion Pipeline initialized")

        # Stage 2: Hierarchical Chunking
        self.chunker = HierarchicalChunkingPipeline()
        log.info("✓ Hierarchical Chunking Pipeline initialized")

        # Stage 3: Embedding Generation
        embedding_config = EmbeddingConfig(
            model_name=RAGConfig.EMBEDDING_MODEL,

```

```

        fallback_model=RAGConfig.EMBEDDING_FALLBACK,
        batch_size=RAGConfig.BATCH_SIZE,
    )
    self.embedder = EmbeddingPipeline(embedding_config)
    log.info(f"✓ Embedding Pipeline initialized ({RAGConfig.EMBEDDING_MODEL})")

    # Stage 4: Vector Storage
    storage_config = StorageConfig(
        host=RAGConfig.TYPESENSE_HOST,
        port=RAGConfig.TYPESENSE_PORT,
        protocol=RAGConfig.TYPESENSE_PROTOCOL,
        api_key=RAGConfig.TYPESENSE_API_KEY,
        batch_size=RAGConfig.BATCH_SIZE,
    )
    self.storage = VectorStoragePipeline(storage_config)
    log.info("✓ Vector Storage Pipeline initialized (Typesense)")

    # Stage 5: LLM
    llm_config = LLMConfig(
        provider="groq",
        api_key=RAGConfig.GROQ_API_KEY,
        model_name=RAGConfig.LLM_MODEL,
        temperature=RAGConfig.LLM_TEMPERATURE,
        max_tokens=RAGConfig.LLM_MAX_TOKENS,
        context_score_threshold=RAGConfig.CONTEXT_THRESHOLD,
        max_context_length=RAGConfig.MAX_CONTEXT_LENGTH,
    )
    self.llm = LLMPipeline(llm_config, self.embedder, self.storage)
    log.info(f"✓ LLM Pipeline initialized ({RAGConfig.LLM_MODEL})")

    log.info("All pipelines initialized successfully!")

except Exception as e:
    log.error(f"Pipeline initialization failed: {e}")
    raise

def get_conversation(self, conversation_id: str):
    """Get or create conversation."""
    if conversation_id not in self.conversations:
        self.conversations[conversation_id] = {
            "history": [],
            "llm_instance": None, # Each conversation has its own LLM instance
        }

```

```

        return self.conversations[conversation_id]

def clear_conversation(self, conversation_id: str):
    """Clear conversation history."""
    if conversation_id in self.conversations:
        del self.conversations[conversation_id]

def health_check(self) -> dict:
    """Check health of all pipelines."""
    return {
        "ingestion": self.ingestion is not None,
        "chunker": self.chunker is not None,
        "embedder": self.embedder is not None and self.embedder.is_ready,
        "storage": self.storage is not None and self.storage.ping(),
        "llm": self.llm is not None,
    }

# Global instance
pipeline_manager = PipelineManager()

def get_pipeline_manager() -> PipelineManager:
    """Dependency injection for FastAPI."""
    return pipeline_manager

```

schemas/models.py

```

"""
API Schemas
-----
Pydantic models for request/response validation.
"""

from pydantic import BaseModel, Field
from typing import List, Optional, Dict, Any
from enum import Enum

#

```

```
# Enums
```

```
#
```

```
class DocumentStatus(str, Enum):
```

```
    PENDING = "pending"
```

```
    PROCESSING = "processing"
```

```
    COMPLETED = "completed"
```

```
    FAILED = "failed"
```

```
class QueryTypeEnum(str, Enum):
```

```
    NEW = "new"
```

```
    FOLLOWUP = "followup"
```

```
    CLARIFICATION = "clarification"
```

```
class ContextQualityEnum(str, Enum):
```

```
    HIGH = "high"
```

```
    MEDIUM = "medium"
```

```
    LOW = "low"
```

```
    NO_CONTEXT = "none"
```

```
#
```

```
# Document Upload & Processing
```

```
#
```

```
class DocumentUploadResponse(BaseModel):
```

```
    """Response after uploading a document."""
```

```
    document_id: str
```

```
    filename: str
```

```
    status: DocumentStatus
```

```
    message: str
```

```
class DocumentProcessRequest(BaseModel):
```

```
    """Request to process an uploaded document."""
```

```
    collection_name: str = Field(..., description="Target collection name")
```

```
model_version: Optional[str] = Field("v1.0", description="Embedding model version")
```

```
class DocumentProcessResponse(BaseModel):
```

```
    """Response after document processing."""
```

```
    document_id: str
```

```
    filename: str
```

```
    status: DocumentStatus
```

```
    # Processing stats
```

```
    total_pages: int
```

```
    native_pages: int
```

```
    ocr_pages: int
```

```
    total_chunks: int
```

```
    chunks_indexed: int
```

```
    # Performance
```

```
    processing_time_sec: float
```

```
    # Errors
```

```
    errors: List[str] = []
```

```
class DocumentStatusResponse(BaseModel):
```

```
    """Document processing status."""
```

```
    document_id: str
```

```
    filename: str
```

```
    status: DocumentStatus
```

```
    progress_percentage: int
```

```
    current_stage: str
```

```
    message: str
```

```
#
```

```
# Collection Management
```

```
#
```

```
class CollectionCreateRequest(BaseModel):
```

```
    """Request to create a collection."""
```

```
    name: str = Field(..., description="Collection name")
```

```

embedding_dim: int = Field(1024, description="Embedding dimension")
model_name: str = Field("BAAI/bge-large-en-v1.5", description="Embedding model")
drop_if_exists: bool = Field(False, description="Drop existing collection")

```

```

class CollectionCreateResponse(BaseModel):
    """Response after creating collection."""
    name: str
    embedding_dim: int
    model_name: str
    created: bool
    message: str

```

```

class CollectionInfo(BaseModel):
    """Collection information."""
    name: str
    num_documents: int
    embedding_dim: int

```

```

class CollectionListResponse(BaseModel):
    """List of collections."""
    collections: List[CollectionInfo]
    total: int

```

```
#
```

```
# Query & Search
```

```
#
```

```

class QueryRequest(BaseModel):
    """Request to query the RAG system."""
    query: str = Field(..., description="User question")
    collection_name: str = Field(..., description="Collection to search")
    conversation_id: Optional[str] = Field(None, description="Conversation ID for multi-turn")
    k: Optional[int] = Field(5, description="Number of contexts to retrieve")
    filter_by: Optional[str] = Field(None, description="Typesense filter expression")

```

```

class CitationSchema(BaseModel):
    """Citation information."""
    source_file: str
    hierarchy: List[str]
    page_numbers: List[int]
    score: float

class QueryResponse(BaseModel):
    """Response from RAG query."""
    answer: str
    query: str
    conversation_id: str

    # Query metadata
    query_type: QueryTypeEnum
    context_quality: ContextQualityEnum

    # Citations
    citations: List[str]
    contexts_used: int
    best_score: float

    # Performance
    response_time_sec: float
    prompt_tokens: int
    completion_tokens: int
    total_tokens: int

    # Model info
    model_name: str

```

```

class ConversationHistoryResponse(BaseModel):
    """Conversation history."""
    conversation_id: str
    messages: List[Dict[str, str]]
    total_messages: int

```

```

#

```

```

# Health & Status

```

#

```
class HealthResponse(BaseModel):
```

```
    """API health status."""
```

```
    status: str
```

```
    version: str
```

```
    pipelines: Dict[str, bool]
```

```
    message: str
```

```
class SystemStatusResponse(BaseModel):
```

```
    """Detailed system status."""
```

```
    api_version: str
```

```
    # Pipeline status
```

```
    ingestion_ready: bool
```

```
    chunker_ready: bool
```

```
    embedder_ready: bool
```

```
    storage_ready: bool
```

```
    llm_ready: bool
```

```
    # Storage info
```

```
    total_collections: int
```

```
    total_documents: int
```

```
    # Model info
```

```
    embedding_model: str
```

```
    embedding_dim: int
```

```
    llm_model: str
```

#

```
# Error Responses
```

```
#
```

```
class ErrorResponse(BaseModel):
```

```
    """Error response."""
```

```
    error: str
```

```
detail: str
status_code: int
```

Frontend:

App.jsx

```
import { useState } from 'react'
import logo from './assets/logo.jpg'
import viteLogo from '/vite.svg'
import './App.css'
import UploadPanel from './components/UploadPanel'
import ChatPanel from './components/ChatPanel'

function App() {
  const [ready, setReady] = useState(false);
  const [documents, setDocuments] = useState([]);

  return (
    <div className="main-container">
      <div className="title">
        <img src={logo} className="logo" />
        <div className="title-text">
          <h1>LEXORA</h1>
          <h3>Legal Expert Retrieval & Explainable Reasoning Assistant</h3>
        </div>
      </div>

      <div className='container'>
        <div className='sidebar'>
          <UploadPanel setReady={setReady} documents={documents}
setDocuments={setDocuments} />
        </div>

        <div className='chat'>
          <ChatPanel ready={ready} />
        </div>
      </div>
    </div>
  )
}
```

```
)  
}
```

```
export default App
```

Components/ChatPanel.jsx

```
import React, { useState, useRef, useEffect } from "react";  
import { queryRAG } from "../api";  
import ReactMarkdown from "react-markdown";  
  
export default function ChatPanel({ ready }) {  
  const [messages, setMessages] = useState([]);  
  const [input, setInput] = useState("");  
  const [conversationId, setConversationId] = useState(null);  
  const [loading, setLoading] = useState(false);  
  const [error, setError] = useState("");  
  
  const bottomRef = useRef(null);  
  
  // auto scroll  
  useEffect(() => {  
    bottomRef.current?.scrollIntoView({ behavior: "smooth" });  
  }, [messages, loading]);  
  
  const sendMessage = async () => {  
    if (!input.trim() || loading || !ready) return;  
  
    const userMessage = { role: "user", text: input };  
  
    setMessages((prev) => [...prev, userMessage]);  
    setInput("");  
    setLoading(true);  
    setError("");  
  
    try {  
      const res = await queryRAG(input, conversationId);  
  
      setConversationId(res.conversation_id);  
  
      const assistantMessage = {  
        role: "assistant",  
        text: res.answer,  

```

```

};

setMessages((prev) => [...prev, assistantMessage]);
} catch (err) {
  console.error(err);
  setError("Failed to get response. Please try again.");
} finally {
  setLoading(false);
}
};

return (
  <div className="chat-container">

    <div className="messages">
      {!ready && (
        <p style={{ color: "grey" }}>
          Upload and process a document to start chatting.
        </p>
      )}

      {messages.map((m, i) => (
        <div
          key={i}
          className={`message ${m.role === "user" ? "user" : "assistant"}`}
        >
          <ReactMarkdown>{m.text}</ReactMarkdown>
        </div>
      ))}

      {loading && (
        <div className="message assistant">Assistant is thinking...</div>
      )}

      {error && (
        <div style={{ color: "red" }}>{error}</div>
      )}

      <div ref={bottomRef} />
    </div>

    <div className="chat-input">
      <input

```

```

    value={input}
    disabled={!ready || loading}
    onChange={(e) => setInput(e.target.value)}
    onKeyDown={(e) => e.key === "Enter" && sendMessage()}
    placeholder="Ask about the document..."
  />

  <button
    onClick={sendMessage}
    disabled={!input.trim() || loading}
  >
    {loading ? "Sending..." : "Send"}
  </button>
</div>
</div>
);
}

```

Components/UploadPanel.jsx

```

import React, { useState } from "react";
import { uploadDocument, processDocument } from "../api";

export default function UploadPanel({ setReady, documents, setDocuments }) {
  const [file, setFile] = useState(null);
  const [uploading, setUploading] = useState(false);
  const [error, setError] = useState("");

  const handleUpload = async () => {
    if (!file) {
      setError("Select a file first");
      return;
    }

    try {
      setUploading(true);
      setError("");

      const upload = await uploadDocument(file);
      const docId = upload.document_id;

      await processDocument(docId);
    }
  }
}

```

```

setDocuments((prev) => [...prev, file.name]);

setReady(true);
setFile(null);
} catch (err) {
  setError("Upload failed");
} finally {
  setUploading(false);
}
};

return (
  <div>
    <h3>Documents</h3>

    <ul>
      {documents.map((doc, i) => (
        <li key={i} style={{ listStyleType: "none" }}> {doc}</li>
      ))}
    </ul>

    <input
      type="file"
      disabled={uploading}
      onChange={(e) => setFile(e.target.files[0])}
    />

    {error && <p style={{ color: "red" }}>{error}</p>}
    <br /><br /><br />
    <button onClick={handleUpload} disabled={!file || uploading}>
      {uploading ? "Processing..." : "Upload"}
    </button>
  </div>
);
}

```

Api.js

```

const api = "http://localhost:8000";

export async function uploadDocument(file) {
  const formData = new FormData();
  formData.append("file",file);

```

```

const res = await fetch(`${api}/documents/upload`, {
  method: "POST",
  body: formData,
});
return res.json();
}

export async function processDocument(documentId) {
  const res = await fetch(`${api}/documents/${documentId}/process`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({ collection_name: "legal_docs",
      model_version: "v1",
    }),
  });

  return res.json();
}

export async function queryRAG(query, conversationId) {
  const res = await fetch(`${api}/query/`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      query,
      collection_name: "legal_docs",
      conversation_id: conversationId,
      k: 5,
    }),
  });

  return res.json();
}

```